Web Application Attack Detection (SQLi/XSS) Using ML on HTTP Request Patterns

DEPARTMENT:
CSE(CYS)

NAME&ROLLNO:
Navya sree(CB.SC.U4CYS24049)
Sriya sudhikshana(CB.SC.U4CYS24032)
Kandavalli.mokshitha(CB.SC.U4CYS24026)
Srimathi(CB.SC.U4CYS24070)
Nikhil(CB.SC.U4CYS24011)

Problem definition

- Web applications are vulnerable to SQL Injection and XSS attacks.
- Manual detection of attack payloads is difficult.
- A machine learning system is developed to analyze HTTP requests and detect attack patterns.

Type of ML Task

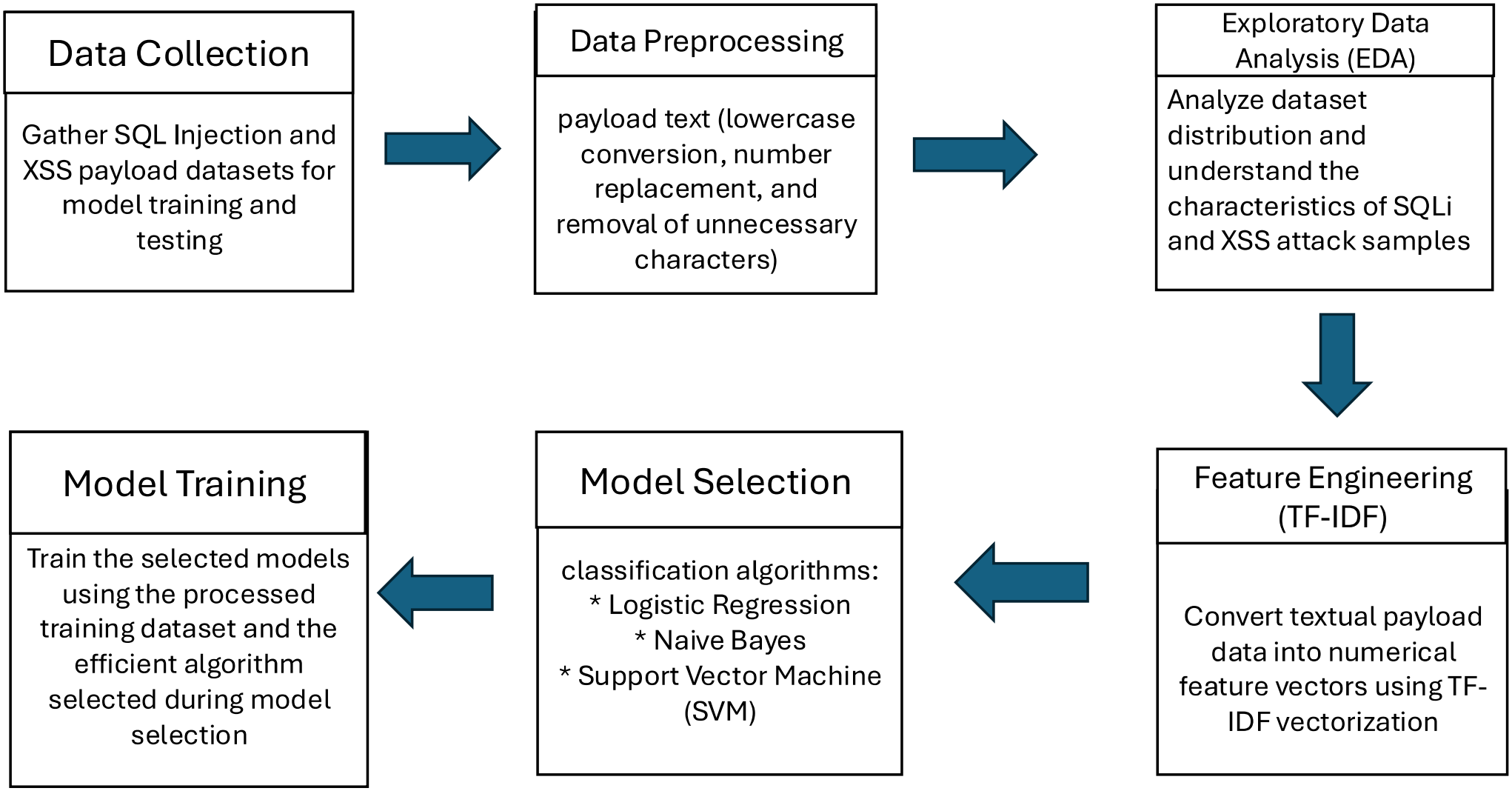
- Classification (Binary Classification)

- The model predicts categorical labels:
- SQLi or XSS.

Objective of the Model

- Automatically detect SQL Injection and XSS attacks from HTTP requests using machine learning techniques.

Machine Learning Project Workflow



Model Evaluation

Evaluate model performance using metrics such as Accuracy, Precision, Recall, F1-Score, and Confusion Matrix

Interpretation

Analyze prediction results and determine the best performing model

Conclusion

Deploy the final model to detect SQL Injection and XSS attacks through a Flask-based web application

Dataset Overview

1 Dataset Source

- Collected from **Kaggle** & **GitHub**
- Public security datasets
- Contains SQL Injection & XSS payloads
- Merged using `prepare_data.py`

2 Number of Records

Training Set: 49,719
Testing Set: 2,569
Training Class Distribution:

- SQLi → 35,682
- XSS → 14,037

3 Features

- Primary Column: `payload`
- Text converted using TF-IDF
- Max Features: 5000
- N-gram Range: (1,2)

4 Target Variable

- Column: `label`
- Classes:
  - SQLi
  - XSS
- Binary Classification Problem

Total Samples: 52,288 | Feature Space: 5000 TF-IDF Features

Data Preprocessing and Feature Engineering

Missing Value Handling	Encoding Techniques
.The dataset was checked for missing values in the payload column, which contains the HTTP request or attack payload text.	.The target variable label represents the type of web attack. These labels are used by the machine learning models during training to learn the relationship between the input payload and the attack category.
.During inspection, no significant missing values were found in the dataset, so no rows needed to be removed or imputed.	.Feature Scaling Traditional feature scaling methods such as normalization or standardization were not required in this project.
.In case any missing values appear, they are automatically handled by converting the payload into a string during preprocessing.	.This is because the features are generated using TF-IDF vectorization, which already produces normalized numerical values representing the importance of words in the payload text.

Feature Engineering

Text payloads cannot be directly used by machine learning models, so they were converted into numerical feature vectors using TF-IDF (Term Frequency–Inverse Document Frequency).

Configuration used in the project:

Maximum Features: 5000
N-gram Range: (1,2)
Additionally, preprocessing techniques were applied to improve pattern detection:
Convert all text to lowercase
Replace numeric values with the token NUM
Remove unnecessary special characters
Extract meaningful parts from HTTP requests
These steps help the model identify important patterns related to SQL Injection and XSS attacks.

Exploratory Data Analysis (EDA)

Key Patterns in Payloads

SQL Injection (SQLi)

- Common keywords: `OR`, `UNION`, `SELECT`, `WHERE`
- Logical conditions (1=1)
- Database query structures

Cross-Site Scripting (XSS)

- HTML/JS tags: `<script>`
- Functions: `alert()`
- Event handlers: `onerror`, `onclick`

Feature Relationship Method

Because the data is textual:

- No traditional correlation matrix
- Used **TF-IDF** for feature extraction
- Words strongly related to attacks get higher weights
- Helps model differentiate SQLi vs XSS

Class Distribution

Training Data:

- SQLi → 35,682 (72%)
- XSS → 14,037 (28%)

Algorithms Used & Model Selection

1 Algorithms Evaluated

- Logistic Regression
- Multinomial Naive Bayes
- Linear SVM

✓ Suitable for TF-IDF based text classification

✓ Compared using accuracy & performance metrics

2 Final Model Selected

->Multinomial Naive Bayes

Why selected?

- Works well with high-dimensional sparse TF-IDF data
- Fast training & prediction
- Low computational cost
- Effective for text classification (e.g., spam detection)

3 Algorithm Assumptions

Based on **Bayes' Theorem**

Main assumptions:

- Features are conditionally independent
- Class probability depends on feature frequency
- Model predicts class with highest probability

FORMULA:

$$P(\text{Class} \mid \text{Features}) \propto P(\text{Features} \mid \text{Class}) \times P(\text{Class})$$

Model Training & Deployment Workflow

Raw Payload Data <---------- 1

80:20 Stratified Split <---------- 2

TF-IDF Vectorization
(Max Features: 5000 | N-grams: 1–2) <---------- 3

Multinomial Naive Bayes Training
(Default Parameters) <---------- 4

Model Saved (.pkl files) <---------- 5

Flask Web Application <---------- 6

Real-Time SQLi/XSS Detection <---------- 7

MODEL COMPARISION

Model	Strengths	Limitations	Performance Observation
Logistic Regression	Simple and interpretable linear classifier	Can struggle with complex text patterns	Performed reasonably well but slightly slower for large feature sets
Multinomial Naive Bayes	Very efficient for text classification and works well with TF-IDF features	Assumes feature independence	Achieved high accuracy with fast training time
Linear Support Vector Machine (SVM)	Powerful for high-dimensional text data	Higher computational cost	Good performance but slower compared to Naive Bayes

Best Model Selection

Selected Model: Multinomial Naive Bayes

Reason	Explanation
Efficiency	Fast training and prediction even with large TF-IDF feature vectors
Suitable for Text Data	Performs well with word frequency based features
Simplicity	Requires fewer computational resources
Strong Performance	Achieved high accuracy for detecting SQLi and XSS attacks

Limitations

Data Limitations

The dataset contains only SQL Injection (SQLi) and Cross-Site Scripting (XSS) attack samples.
It does not include normal (benign) HTTP requests, which limits the model's ability to distinguish between normal traffic and attacks.

Model Limitations

The Naive Bayes model assumes that all features are independent, which may not always be true for real-world attack patterns.
Although the model performs well for text classification, it may struggle with complex or obfuscated attack payloads.

Assumptions Affecting Performance

The model relies on textual patterns extracted from HTTP payloads using TF-IDF features.
If attackers modify payload structures or use advanced obfuscation techniques, the model's detection accuracy may decrease.

Conclusion

Final Outcome

A machine learning model was developed to detect SQL Injection (SQLi) and Cross-Site Scripting (XSS) attacks from HTTP requests using TF-IDF features and a Naive Bayes classifier.
A Flask web interface was also implemented to allow users to test HTTP inputs and detect potential attacks

Key Takeaways

* Machine learning can effectively detect web attack patterns from textual data.
* TF-IDF helps convert payload text into numerical features.
* Naive Bayes provides fast and efficient text classification.

Future Scope

* Include normal HTTP traffic to improve real-world detection.
* Extend detection to more web attacks.
* Explore advanced ML or deep learning models for improved accuracy.

THANK YOU